

AGENTIUM V2 — COMPLETE ARCHITECTURE & FUNCTION DOCUMENTATION

Context & Trust Integrity Toolkit for Multi-Agent and Long-Running AI Systems

Version: 2.0.3	Core Dependency Pledge: 0 Third-Party Dependencies (Python 3.11+ stdlib only)
Testing Status: 21/21 Standalone Suites & 57/57 Unit Tests PASS (100%)	Target Environment: Production Multi-Agent Systems & CI/CD
Scope: All 10 Features (F1-F10) & 3 Framework Adapters	Backward Compatibility: 100% v1 Compatibility Shim via agentium._v1

1. ARCHITECTURAL OVERVIEW & ZERO-DEPENDENCY GUARANTEE

Agentium v2 solves the four existential failures of enterprise AI agent workflows: **(1) Silent context loss** during context-window compaction, **(2) Hallucinated handoff drift** between collaborating agents, **(3) Accidental destructive mutations** without authorization, and **(4) Secret credential leaks** into logs. Crucially, the entire core runtime uses **Standard Library only** (tomllib, hashlib, unicodedata, contextvars, dataclasses, argparse), guaranteeing sub-millisecond cold starts and zero supply chain vulnerability risk.

#	Feature	Key Symbols	Functional Responsibility & Safety Invariants
F1	Claims & Evidence-Based Trust	Claim, ClaimStore, EvidenceRef	Formal claim verification with Peer Agreement Defense: peer agents cannot self-certify ungrounded statements without external evidence.
F2	Multi-Agent Lineage Graph	LineageGraph, BlameReport, diff_lineage	Pure Python stdlib DAG (zero networkx). Kahn's cycle detection, root-cause blame reports, and Mermaid flowchart generation.
F3	Handoff Contract	HandoffPacket, lint_handoff, enforce_handoff_size	Structured inter-agent packets with hard token limits. Inviolable invariant: constraints can never be dropped under any policy.
F4	Action Gate	ActionGate, guarded, ActionBlockedError	Tool safety barrier (read, write, destructive). Destructive tools require human confirmation or verified claim. Supports shadow/enforce modes.
F5	Context Pins	Pin, PinStore, reinject, guard_compaction	Critical instructions, boundaries, and SLAs survive conversation compaction via deterministic, idempotent re-injection.
F6	Compaction Soak Harness	soak, TruncateCompactor, LastNCompactor	Simulates 20+ turn multi-agent conversations under aggressive compaction to verify zero pin loss across compactors.
F7	Run Fingerprint	Fingerprint, fingerprint	Canonical SHA-256 hashing of models, system prompts, tool schemas, and extras with JSON-path selector exclusion (\$.params.seed).
F8	Drift CI Gate	LockFile, diff_fingerprints, agentium lock/check	Baseline schema locking and pull-request drift check. GitHub Composite Action posts markdown diffs and enforces gates.
F9	Developer Experience	agentium init, agentium doctor	Static framework scanner (no user code execution) generating agentium.toml, plus 9-point system and lineage health diagnostics.
F10	Speculative Prefetch	PrefetchManager, ToolPredictor	Experimental Markov transition tool predictor and TTL cache. Strictly confined to read-only tools with circuit-breaker auto-shutdown.

2. DETAILED FUNCTION-BY-FUNCTION SPECIFICATION

F1: Claims & Evidence-Based Trust (agentium.lineage.claims)

Enforces epistemic rigor across multi-agent pipelines.

- `ClaimStore.add(statement, confidence=0.5, evidence=None, dependencies=None, source_agent=None) -> Claim`
Creates a new unverified claim. Emits `EventType.CLAIM`.
- `ClaimStore.record_peer_agreement(claim_id, peer_agent) -> Claim`
CRITICAL DEFENSE: Records peer agreement in metadata but leaves status as 'unverified'. Peer agreement cannot upgrade trust without external evidence.
- `ClaimStore.verify(claim_id, verifier_name, evidence) -> Claim`
Upgrades claim status to 'verified' with attached grounding `EvidenceRef` (tool result, DB query).
- `ClaimStore.refute(claim_id, refuter_name, reason) -> Claim`
Marks claim as 'refuted', sets confidence to 0.0, and records reason.

Primary Scenarios: Multi-agent collaborative pipelines where downstream agents must not act on ungrounded hallucinations from upstream agents.

F2: Multi-Agent Lineage Graph (agentium.lineage.graph)

Pure stdlib DAG tracking multi-agent provenance and root cause analysis.

- `LineageGraph.add_node(id, type, label) / add_edge(src, dst, relation)`
Constructs DAG of agents, claims, tools, and handoffs.
- `LineageGraph.has_cycle() -> bool`
Kahn's topological sort detecting cycles in delegation and claim dependencies.
- `LineageGraph.blame(claim_id) -> BlameReport`
Traces claim back to origin agent, tool calls, and dependencies, reporting whether grounded.
- `LineageGraph.to_mermaid() -> str`
Renders beautiful Mermaid flowchart markdown syntax for documentation.
- `diff_lineage(graph_a, graph_b) -> LineageDiff`
Computes topological differences in claims and handoffs between two runs.

Primary Scenarios: Debugging complex multi-agent failures, audit logging for compliance, and visual inspection of agent interactions.

F3: Handoff Contract (agentium.lineage.handoff)

Structured inter-agent task delegation protocol.

- `HandoffPacket(from_agent, to_agent, task, claims, constraints, max_tokens)`
Encapsulates task context, verified claims, and operational boundaries.
- `enforce_handoff_size(packet, on_overflow='truncate_unverified') -> HandoffPacket`
CRITICAL INVARIANT: Constraints are NEVER dropped under any policy. If constraints alone exceed budget, raises `HandoffOverflowError`.
- `lint_handoff(packet) -> List[LintIssue]`
Audits packet for circular delegations (self-handoff), empty task, and ungrounded high-confidence claims.

Primary Scenarios: Enterprise workflows delegating critical tasks between specialized models (e.g. planner to coder to reviewer).

F4: Action Gate (agentium.lineage.gate)

Side-effect governance preventing unauthorized mutations.

- `ActionGate.evaluate(tool_name, effect, claims, human_confirmed) -> (bool, str)`
Permits 'read' unconditionally; requires verified claim (≥ 0.9) or human confirmation for 'destructive'.
- `ActionGate.execute_guarded(fn, effect, *args, **kwargs)`
Blocks with `ActionBlockedError` in enforce mode; logs shadow blocks in shadow mode.
- `@guarded(gate, effect='destructive')`
Decorator wrapping any tool or function with `ActionGate` policy evaluation.

Primary Scenarios: Protecting production databases, payment gateways, and APIs from accidental or hallucinated agent destruction.

F5 & F6: Context Pins & Soak Testing (agentium.pin)

Guarantees that critical directives survive context window compaction.

- PinStore.add(key, text, source='system') -> Pin
Registers logical pin. Older pin with same key is marked superseded_by (audit preserved).
- reinject(messages, pin_store, position='end') -> List[messages]
Pure, idempotent function injecting active pins. No-op if identical pins already present.
- guard_compaction(orig_msgs, compacted_msgs, pin_store) -> (repaired, lost, restored)
Detects dropped pins during compaction, re-injects them, and logs compaction event.
- soak(compact, num_turns=20, pin_interval=3) -> SoakReport
Executes multi-turn conversation exercising compaction and verifies pin survival.

Primary Scenarios: Long-running autonomous agents (e.g., coding, research, customer service) where system rules must never be dropped.

F7 & F8: Run Fingerprint & Drift CI Gate (agentium.fingerprint)

Deterministic schema hashing and continuous integration drift gating.

- fingerprint(model, params, system_prompt, tools, ignore) -> Fingerprint
Computes SHA-256 hash across components. Distinguishes structural schema changes vs description edits.
- write_lock_file(path, fp) / read_lock_file(path) -> LockFile
Manages agentium.lock file recording baseline state.
- diff_fingerprints(baseline, current) -> DriftReport
Detects drift, tool additions, tool removals, and generates markdown reports.

Primary Scenarios: CI/CD pipelines to prevent unexpected prompt modifications, model version changes, or tool signature drift.

F9: Developer Experience (agentium.cli.init_cmd & doctor_cmd)

Project initialization and health inspection.

- run_init(project_root, dry_run, force) -> (code, msg)
Statically detects installed frameworks without executing code, generating tailored agentium.toml.
- run_doctor(project_root, strict, check_lock) -> (code, msg)
Runs 9 health checks: Python >= 3.11, config validity, event directory writability, lock status, pin survival, lineage integrity, action gate mode, extras, external scanners.

Primary Scenarios: Onboarding new projects and pre-commit environment health verification.

F10: Speculative Read-Only Prefetch (agentium.speed.prefetch)

Experimental acceleration engine for read-heavy agent workflows.

- PrefetchManager.maybe_prefetch(tool_name, tool_fn, effect, kwargs) -> bool
CRITICAL INVARIANT: NEVER prefetches non-read tools. Rejects write/destructive immediately.
- PrefetchManager.get_or_record_usage(tool_name, kwargs) -> (is_hit, result)
Consumes cached speculative result, avoiding duplicate execution.
- PrefetchManager.reap_wasted()
Purges expired TTL entries; automatically triggers circuit breaker if hit rate < 30% or wasted > 5.
- ToolPredictor.predict_next(current_tool) -> Optional[str]
Markov transition frequency learner predicting next tool call.

Primary Scenarios: Accelerating agent latency in multi-turn data retrieval or document search workflows.

3. FRAMEWORK ADAPTERS

Agentium provides lazy-loaded adapters in agentium.adapters.* for major agent frameworks. Missing framework libraries never break package import; invoking an adapter without the extra installed raises an actionable ImportError with installation instructions:

Adapter	Module	Integration Capabilities	Install Command
LangGraph	LangGraphAdapter	Node execution tracing, state message pin re-injection on checkpoints, event emission.	pip install 'agentium[langgraph]'
CrewAI	CrewAIAdapter	Agent task delegation handoffs, constraint attachment, claim lineage tracing.	pip install 'agentium[crewai]'
OpenAI Agents	OpenAIAgentsAdapter	Tool execution interception, ActionGate destructive authorization, event logging.	pip install 'agentium[openai-agents]'

4. CLI COMPLETE COMMAND REFERENCE

Command Syntax	Exit Code	Description & Output
agentium init [--dry-run] [--force]	0 / 1	Statically detects frameworks and writes agentium.toml. Exits 1 if exists without --force.
agentium doctor [--strict] [--json]	0 / 1	Executes 9 health checks. Strict mode treats warnings as failures (exit 1).
agentium lock --from <mod:fn>	0 / 2	Generates baseline agentium.lock recording fingerprint of model, prompts, and tools.
agentium check [--format text\ json\ md]	0 / 1 / 2	Compares against lock file. Exit 0 = match; Exit 1 = drift detected; Exit 2 = missing lock.
agentium lineage claims <run_id>	0	Lists all claims recorded in event logs for a given run with status flags.
agentium lineage blame <run_id> <claim_id>	0 / 1	Traces root cause of a claim back to origin agent and tool executions.
agentium lineage diff <run_a> <run_b>	0	Diff's claims and handoff topology between two distinct run executions.
agentium handoff lint <path>	0 / 1 / 2	Lints handoff packet JSON against schema, circular assignments, and token caps.
agentium pins render	0	Renders deterministic formatted string of active context pins.
agentium soak	0 / 1	Runs 20-turn compaction soak test and prints survival metrics.

5. VERIFICATION & MASTER TEST RESULTS

Agentium v2 has been validated across **21 standalone test suites** and **57 unit tests**. All tests execute offline with zero third-party packages in core runtime:

Milestone	Test Suite File	Feature Area	Verdict
M0	test_events.py	JSONL Merging & Event Tracing	PASS
M0	test_canonical_hashing.py	Canonical JSON & Float Normalization	PASS
M0	test_redaction.py	Copy-on-Redact Engine	PASS
M0	test_run_context.py	Async Contextvars & Run ID Scopes	PASS
M0	test_tool_decorator.py	@tool Decorator & Effect Tagging	PASS
M0	test_config.py	tomllib Loader & Env Overrides	PASS
M0	test_zero_deps.py	Strict 0 External Dependencies Verification	PASS
M0	test_otel.py	Lazy OpenTelemetry Exporter	PASS
M0	test_cli_basic.py	CLI Core Flags & Subcommands	PASS
M0	test_v1_backward_compat.py	v1 Shims & DeprecationWarning	PASS
M1	test_fingerprint.py	F7: Run Fingerprint & JSON-path Ignore	PASS
M1	test_drift_gate.py	F8: Drift CI Gate & Diff Reporting	PASS
M2	test_init_doctor.py	F9: agentium init & 9-point doctor	PASS
M3	test_pin_guard.py	F5: Pin Store & Compaction Guard	PASS
M3	test_soak.py	F6: Compaction Soak Test Harness	PASS
M4	test_lineage_claims.py	F1: Claims & Peer Agreement Defense	PASS
M4	test_lineage_graph.py	F2: Lineage Graph DAG & Blame	PASS
M4	test_handoff.py	F3: Handoff Contract & Linter	PASS
M4	test_action_gate.py	F4: Action Gate (Shadow & Enforce)	PASS
M5	test_adapters.py	LangGraph, CrewAI, OpenAI Adapters	PASS
M6	test_prefetch.py	F10: Speculative Prefetch & Markov	PASS